

CS1003

Programming in C

Module - I

Structure of C Program

Documentation section

Link section

Definition section

Global declaration section

main() function section

{

Declaration part

Executable part

}

Subprogram section
(user defined function)

Simple C Program

```
/* A first C Program*/
```

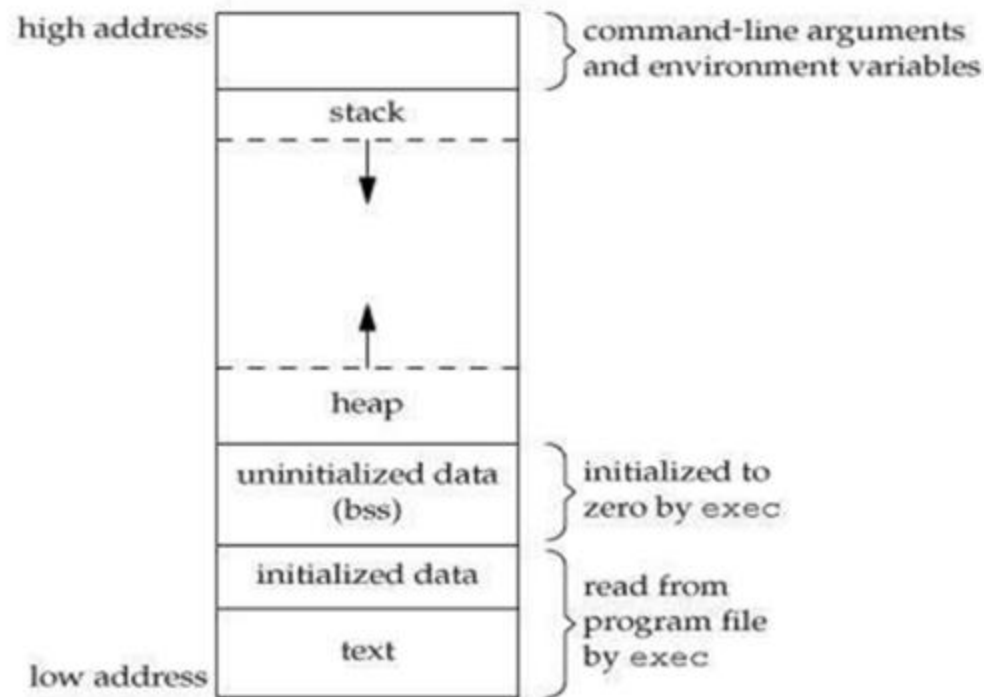
```
#include <stdio.h>
```

```
void main()
```

```
{  
    printf("Hello World \n");  
}
```

Memory Layout of C Program

- This memory layout is organized in following fashion :-



Text or Code Segment

- Text segment contains machine code of the compiled program.
- Usually, the text segment is sharable so that only a single copy needs to be in memory for frequently executed programs, such as text editors, the C compiler, the shells, and so on.
- The text segment of an executable object file is often read-only segment that prevents a program from being accidentally modified.

Initialized Data Segment

- Initialized data stores all global, static, constant, and external variables (declared with extern keyword) that are initialized beforehand. Data segment is not read-only, since the values of the variables can be altered at run time.
- This segment can be further classified into initialized read-only area and initialized read-write area.

Initialized Data Segment - Example

```
#include <stdio.h>

char c[]="rishabh tripathi";    /* global variable stored in Initialized Data Segment in read-
write area*/

const char s[]="HackerEarth";    /* global variable stored in Initialized Data Segment in read-
only area*/

int main()
{
    static int i=11;        /* static variable stored in Initialized Data Segment*/
    return 0;
}
```

Uninitialized Data Segment (bss)

- Data in this segment is initialized to arithmetic 0 before the program starts executing.
- Uninitialized data starts at the end of the data segment and contains all global variables and static variables that are initialized to 0 or do not have explicit initialization in source code.

Uninitialized Data Segment (bss) - Example

```
#include <stdio.h>
char c;      /* Uninitialized variable stored in bss */
int main()
{
    static int i; /* Uninitialized static variable stored in bss */
    return 0;
}
```

Heap

- Heap is the segment where dynamic memory allocation usually takes place.
- When some more memory need to be allocated using malloc and calloc function, heap grows upward.
- The Heap area is shared by all shared libraries and dynamically loaded modules in a process.

Example:-

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char *p=(char*)malloc(sizeof(char));    /* memory allocating in heap segment */
```

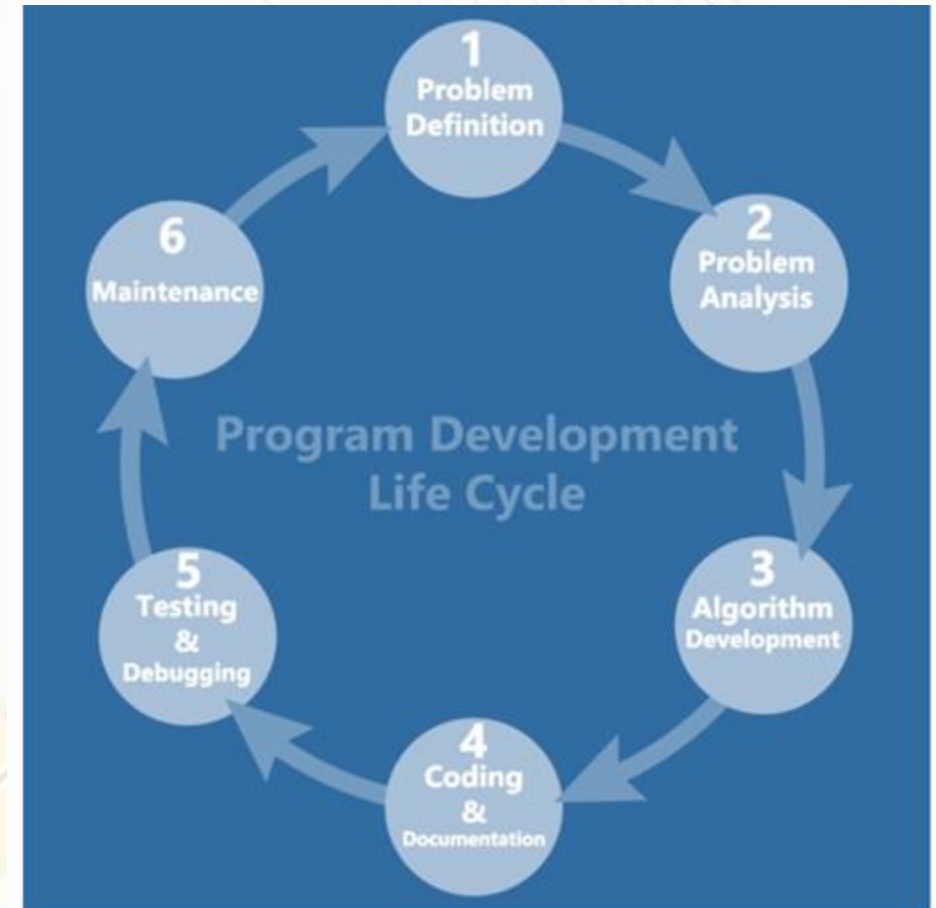
```
    return 0; }
```

Stack

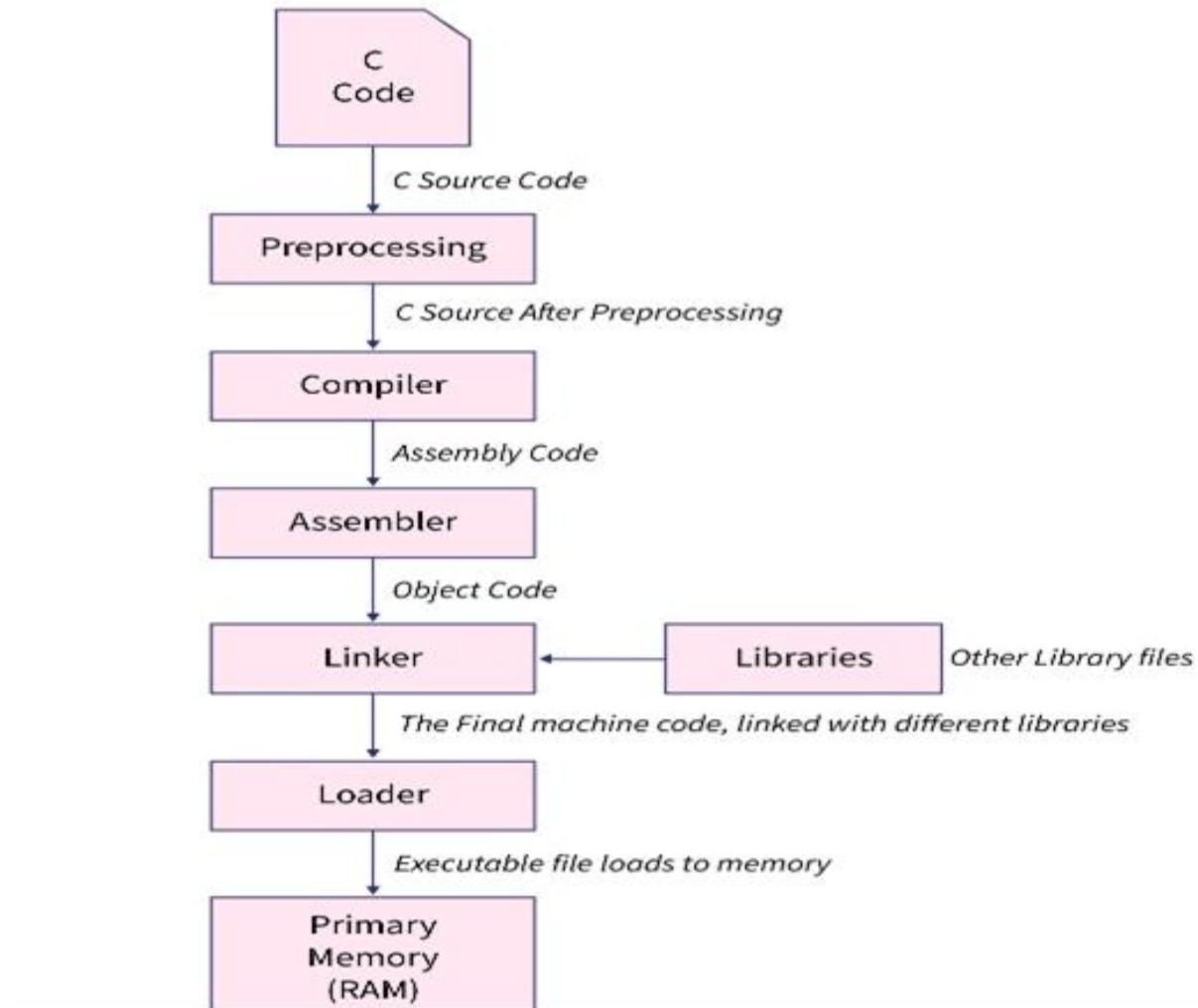
- Stack segment is used to store all local variables and is used for passing arguments to the functions along with the return address of the instruction.
- Which is to be executed after the function call is over.
- Local variables have a scope to the block which they are defined in, they are created when control enters into the block.
- All recursive function calls are added to stack.
- The stack and heap are traditionally located at opposite ends of the process's virtual address space.

Phases of developing program

- Problem Definition.
- Problem Analysis.
- Algorithm Development.
- Coding & Documentation.
- Testing & Debugging.
- Maintenance.



How does a C Program Execute



Cont..

C code: -

When you first write the code in the C language, that source code is sent to the Preprocessing section.

Preprocessing: -

In this section our source code is attached to the preprocessor file.

Different types of header files are used like the `stdio.h`, `math.h`, etc.

`#include, #define` etc,.

Compiler:-

- The preprocessed source code moves to the compiler, and an assembly-level code is generated by the compiler.
- After the compilation of the whole C source code program.
- All the different files which have the C program must be saved with the .c extension.
- Suppose there is a program file named as **first.c**,
- The file first.c will be the source file which will consist of the C source code of the program.

Cont..

- Now, when the file is compiled, the first thing the C compiler does is to search for any error.
- If there is no error, the C compiler will report for no error, after that the compiler will store the file as a **.obj file** of the same name, which is termed as the object file.
- So by this process, the compiler will create the first.obj.
- Although this .obj file will not be executable.
- After the compilation, the process is continued by the assembler section.

Assembler: -

- This part usually generates the Object code, after taking the assembly-level code from the compiler.
- This object code is quite similar to the **machine code** or the set of binary digits.
- After this assembler part, The Linker continues the process, producing an executable.exe file at the end.

Linker:-

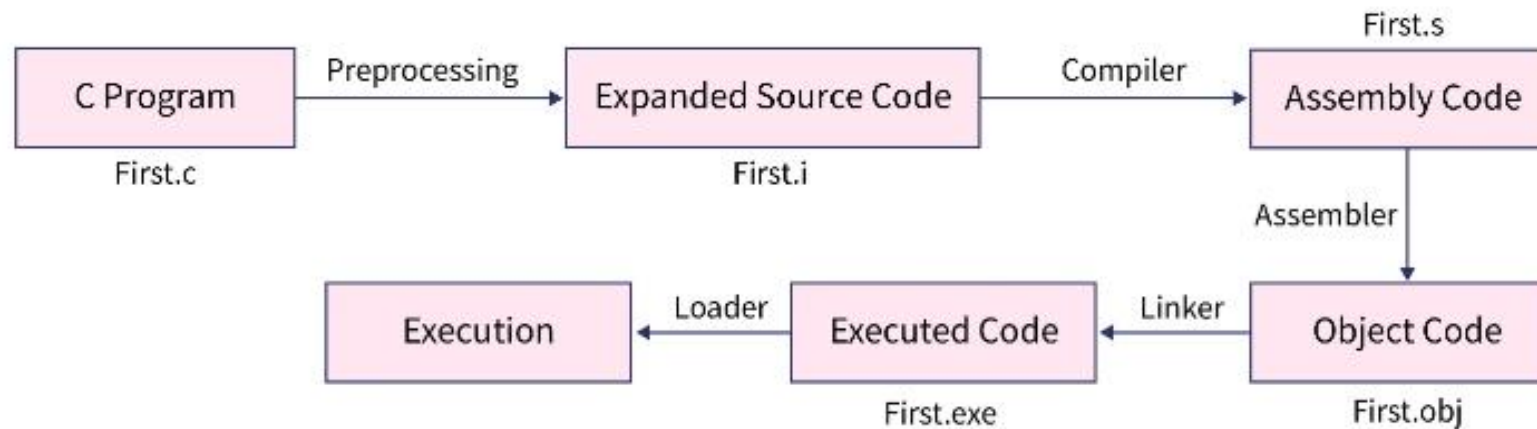
- The basic goal of the linker is to link the object file to the library functions so that the programme may be run as an executable file (.exe).
- In this Linker process, the first.exe file will be created and this file is in an executable format.
- After this process, the next step is the loader process.

Loader:-

- Whenever the command is given for the execution of a particular program,
- The loader plays an important role.
- With the help of the loader, the **.exe file** is loaded in the RAM and the CPU is informed of the starting point of the address of the program where it is loaded.

Syntax :

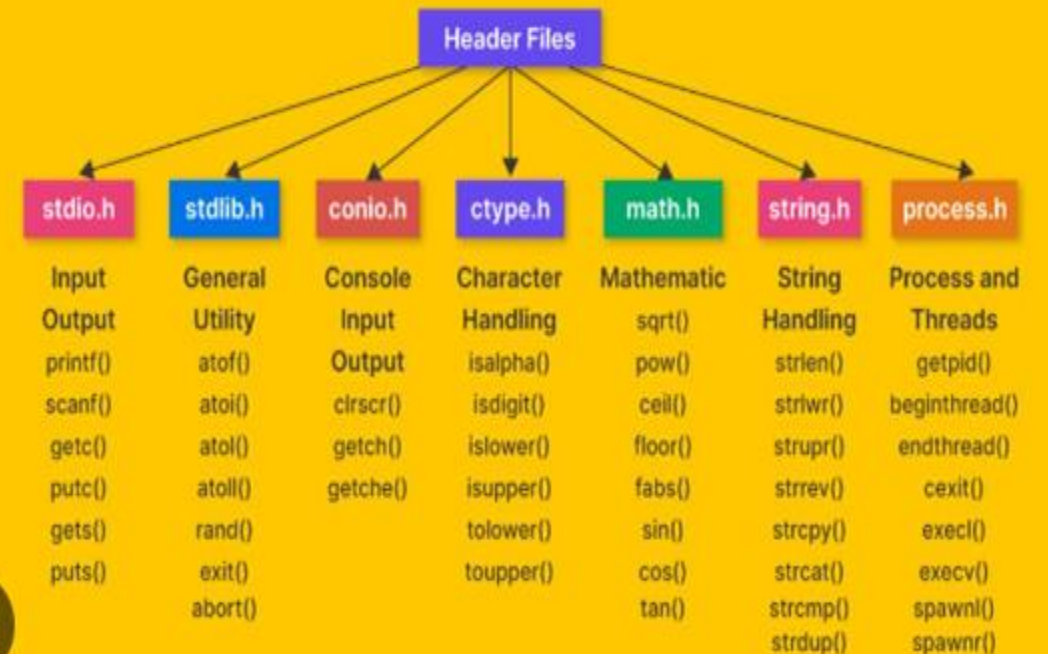
Let us look at the image syntax of the steps of execution of C program.



Library Function

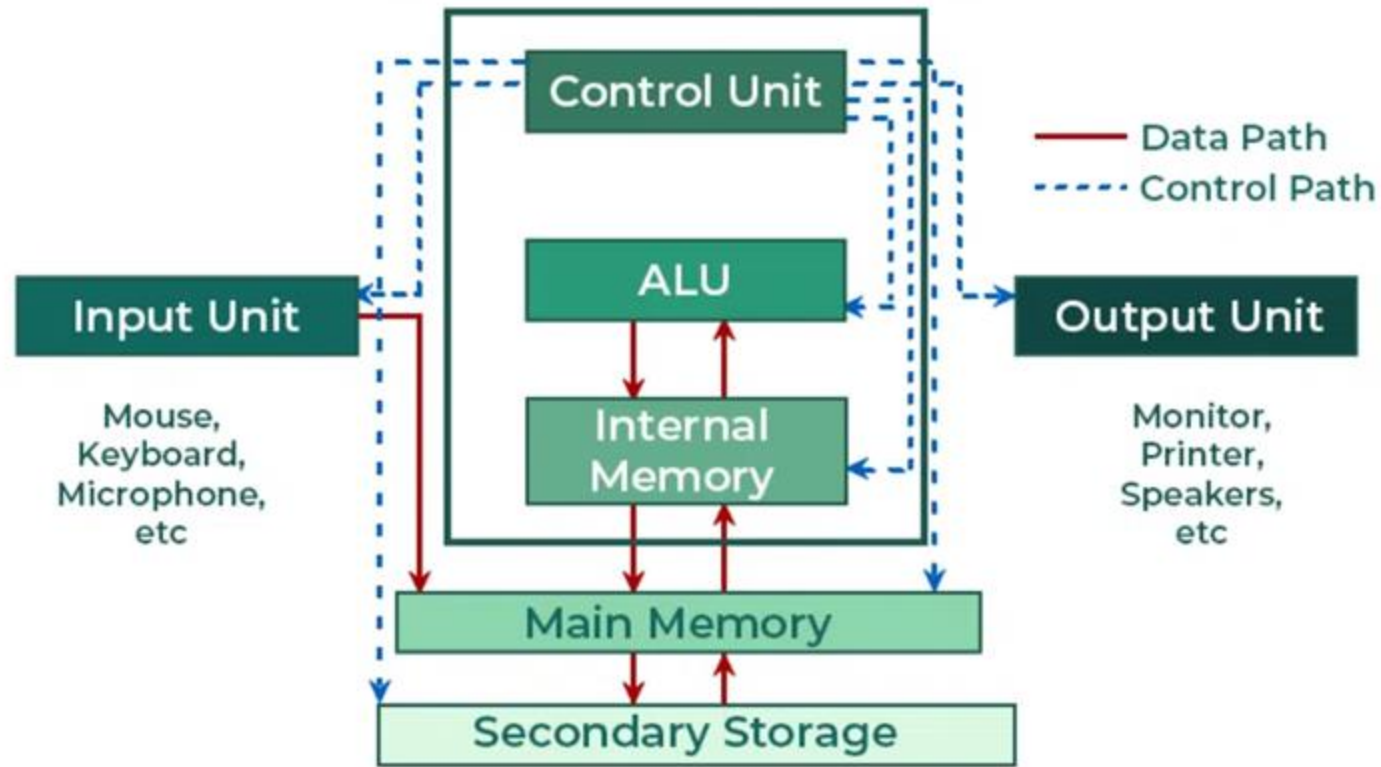
- Library functions are built-in functions that are grouped together and placed in a common location called library.
- Each function here performs a specific operation. We can use this library functions to get the pre-defined output.

All Library Functions In C Explained



Component of Computer

Components of computer



Thank You